Министерство науки и высшего образования Российской Федерации
ФГБОУ ВО «Российский экономический университет имени Г.В. Плеханова»
Пермский филиал

---

# ПРАКТИЧЕСКАЯ РАБОТА №2

*по дисциплине*

**«Поддержка и тестирование программных модулей»**

**Тема: Разработка приложения «Калькулятор»**

Выполнил: студент группы ИПс-11
**Вахрушев Кирилл Дмитриевич**
Проверил: Берестов Д.Б.

Пермь, 2026

## 1. Введение

Настоящая практическая работа посвящена разработке настольного приложения «Калькулятор» на платформе .NET 8 с использованием технологии WPF (Windows Presentation Foundation). Работа выполнена в соответствии с заданием по дисциплине «Поддержка и тестирование программных модулей».

Цель работы — создать приложение-калькулятор, реализующее вычисление арифметических выражений с поддержкой операторов +, -, *, /, ^ (возведение в степень) и скобок. Основная логика вынесена в отдельную библиотеку классов; интерфейс реализован в виде WPF-приложения по паттерну MVVM. Для библиотеки написаны модульные тесты.

## 2. Используемые технологии

В ходе разработки использовались следующие технологии и инструменты:

| Компонент | Технология |
|---|---|
| Платформа | .NET 8 |
| UI-фреймворк | WPF (net8.0-windows) |
| Библиотека логики | Class Library (net8.0) |
| Тестирование | xUnit |
| Сериализация | System.Text.Json |
| IDE | Visual Studio 2022 |
| Система контроля версий | Git / GitHub |

## 3. Структура проекта

Решение состоит из трёх проектов:

- VakhrushevCalculatorLib — библиотека классов с бизнес-логикой
- VakhrushevCalculatorApp — WPF-приложение (интерфейс)
- VakhrushevCalculatorTests — проект модульных тестов (xUnit)

Такое разделение соответствует принципу разделения ответственности: библиотека не зависит от WPF и может быть протестирована изолированно; приложение зависит только от библиотеки и не содержит бизнес-логики.

Внутренняя структура VakhrushevCalculatorLib:

- Calculators/ — алгоритм вычисления выражений (ReversePolishNotation.cs, CalculatorEngine.cs)
- Models/ — модели данных (CalculationHistoryItem.cs, ThemeSettings.cs)
- Services/ — сервисы персистентности (HistoryService.cs, SettingsService.cs)
- Infrastructure/ — вспомогательные классы (AppPaths.cs)

## 4. Создание проекта в Visual Studio

Для создания решения в Visual Studio 2022 был выполнен следующий порядок действий:

1. Создан новый Solution с именем VakhrushevCalculator
2. Добавлен проект Class Library (.NET 8) с именем VakhrushevCalculatorLib
3. Добавлен проект WPF Application (.NET 8) с именем VakhrushevCalculatorApp
4. Добавлен проект xUnit Test Project с именем VakhrushevCalculatorTests
5. Добавлены ссылки на проекты: VakhrushevCalculatorApp → VakhrushevCalculatorLib; VakhrushevCalculatorTests → VakhrushevCalculatorLib

## 5. Реализация библиотеки классов

### 5.1 Алгоритм вычисления выражений

Для разбора и вычисления арифметических выражений реализован алгоритм Shunting-yard (алгоритм сортировочной станции Дейкстры). Алгоритм переводит инфиксную запись выражения в обратную польскую запись (RPN), после чего вычисляет результат через стек операндов.

Реализация разбита на три этапа:

6. Tokenize — токенизация строки: числа, операторы, скобки
7. ToRpn — перевод токенов в обратную польскую запись
8. EvaluateRpn — вычисление RPN-выражения через стек

Публичный метод ReversePolishNotation.Evaluate объединяет все три этапа:

```
public static decimal Evaluate(string expression)
{
    if (expression is null)
        throw new ArgumentNullException(nameof(expression));

    var tokens = Tokenize(expression);
    var rpn    = ToRpn(tokens);
    return EvaluateRpn(rpn);
}
```

Для хранения чисел используется тип decimal вместо double, что исключает ошибки представления с плавающей точкой (например, 0.1 + 0.2 = 0.3, а не 0.30000000000000004). Вся арифметика выполняется в блоке checked — переполнение вызывает OverflowException.

Унарный минус определяется контекстно: символ «-» трактуется как унарный, если предыдущий токен — это оператор, открывающая скобка или начало выражения.

## 5.2 CalculatorEngine

Класс CalculatorEngine является фасадом библиотеки и предоставляет следующий API:

| Метод | Описание |
|---|---|
| Calculate(string) | Вычисление и запись в историю |
| Evaluate(string) | Вычисление без записи в историю (предпросмотр) |
| GetHistory() | Список истории (до 200 записей) |
| ClearHistory() | Очистка истории с сохранением |
| GetThemeSettings() | Получение настроек темы |
| UpdateThemeSettings() | Обновление и сохранение темы |
| SaveAll() | Явное сохранение на диск |

```
public decimal Calculate(string expression)
{
    var normalized = NormalizeExpression(expression);
    var result     = ReversePolishNotation.Evaluate(normalized);
    AddHistory(normalized, result);
    return result;
}

public decimal Evaluate(string expression)
{
    var normalized = NormalizeExpression(expression);
    return ReversePolishNotation.Evaluate(normalized);
}
```

История ограничена 200 записями. При превышении лимита удаляются наиболее старые записи методом RemoveRange.

## 5.3 Модели

Для хранения данных используются две модели:

- CalculationHistoryItem — иммутабельная запись истории (Expression, Result, Timestamp). Сериализуется через [JsonConstructor] без публичных сеттеров.

- ThemeSettings — настройки темы (IsDarkTheme, AccentColor). Содержит статическое поле Default с настройками по умолчанию (тёмная тема, акцент #1E90FF).

```
public sealed class CalculationHistoryItem
{
    [JsonConstructor]
    public CalculationHistoryItem(string expression, decimal result, DateTime
timestamp)
    { Expression = expression; Result = result; Timestamp = timestamp; }
    public string   Expression { get; }
    public decimal  Result     { get; }
    public DateTime Timestamp  { get; }
}

public sealed class ThemeSettings
{
    public static readonly ThemeSettings Default =
        new(isDarkTheme: true, accentColor: "#1E90FF");
    public bool   IsDarkTheme { get; set; } = true;
    public string AccentColor { get; set; } = "#1E90FF";
}
```

## 5.4 Сервисы

Для персистентности данных реализованы два сервиса:

- HistoryService — сохранение и загрузка истории в history.json
- SettingsService — сохранение и загрузка настроек в settings.json

Файлы хранятся в %LocalAppData%\PocketCalc\. Для устранения дублирования пути создан вспомогательный класс AppPaths. JsonSerializerOptions вынесены в статические поля для исключения лишних аллокаций.

```
internal static class AppPaths
{
    private const string AppFolderName = "PocketCalc";

    public static string GetFilePath(string fileName)
    {
        var baseDir = Environment.GetFolderPath(
            Environment.SpecialFolder.LocalApplicationData);
        return Path.Combine(baseDir, AppFolderName, fileName);
    }
}
```

Методы Load() в обоих сервисах возвращают значения по умолчанию при отсутствии файла или ошибке десериализации, что обеспечивает надёжный первый запуск приложения.

# 6. Реализация WPF-приложения

## 6.1 Архитектура MVVM

Приложение построено по паттерну MVVM (Model-View-ViewModel):

- Model — классы библиотеки VakhrushevCalculatorLib
- ViewModel — MainViewModel, реализует INotifyPropertyChanged
- View — MainWindow.xaml с привязками данных

ViewModel не содержит зависимостей на System.Windows.Media — цвета темы хранятся как HEX-строки и преобразуются в Brush только на стороне View через конвертер StringToBrushConverter.

В конструкторе MainViewModel инициализируются пять команд:

```
AppendCommand     = new RelayCommand(p => AppendToken(p as string ??
string.Empty));
ClearAllCommand   = new RelayCommand(_ => ClearAll());
ClearEntryCommand = new RelayCommand(_ => ClearEntry());
BackspaceCommand  = new RelayCommand(_ => Backspace());
EvaluateCommand   = new RelayCommand(_ => EvaluateAndCommit(),
                        _ => !string.IsNullOrWhiteSpace(Expression));
```

## 6.2 Интерфейс

Интерфейс представляет собой компактный калькулятор с дисплеем и клавиатурой кнопок. Дисплей содержит две текстовые области: строку выражения и строку результата (предпросмотр в реальном времени).

Клавиатура имеет иерархию стилей кнопок:

- FuncButtonStyle — служебные кнопки: (, ), CE, C, $x^y$, ⌫
- NumButtonStyle — цифры 0–9 и десятичная точка
- OpButtonStyle — операторы ÷ × − +
- EqualButtonStyle — кнопка =

Цвета кнопок и фона определяются темой. Тёмная тема использует акцентный цвет #3ECFB5 (бирюзовый) для кнопок операторов и равно; светлая — #0D9488.

## 6.3 Возможности приложения

Приложение поддерживает следующие возможности:

- Вычисление арифметических выражений с операторами +, -, *, /, ^ и скобками
- Унарный минус в начале выражения и после оператора
- Предпросмотр результата в реальном времени без нажатия =

- Умная очистка последнего токена (CE) — удаляет последнее введённое число вместе с предшествующим унарным минусом
- Полная очистка выражения (C)
- Переключение тёмной и светлой темы (параметр IsDarkTheme в settings.json)

### 6.4 Ввод с клавиатуры

Помимо мыши, поддерживается ввод с клавиатуры:

| Клавиша | Действие |
|---|---|
| Цифры, операторы | Ввод символа в выражение |
| Точка / запятая | Десятичный разделитель |
| ( и ) | Скобки |
| Enter / Return | Вычисление результата |
| Backspace | Удаление последнего символа |
| Escape | Полная очистка выражения |

### 6.5 Сохранение данных

При закрытии окна вызывается MainViewModel.Persist(), который через CalculatorEngine.SaveAll() сохраняет историю вычислений и настройки темы в JSON-файлы:

- %LocalAppData%\PocketCalc\history.json
- %LocalAppData%\PocketCalc\settings.json

При следующем запуске приложение загружает сохранённые настройки и применяет цветовую схему согласно значению поля IsDarkTheme.

## 7. Модульное тестирование

Для библиотеки VakhrushevCalculatorLib написаны модульные тесты с использованием фреймворка xUnit. Тесты охватывают все основные сценарии работы приложения.

### 7.1 Структура тестов

Тесты организованы в группы с использованием атрибутов [Fact] для одиночных тестов и [Theory] + [InlineData] для параметризованных:

| Группа тестов | Что проверяется |
|---|---|

| | |
|---|---|
| Базовые операции | +, -, *, /, ^ через [Theory] |
| Десятичные числа | Точка и запятая, точность decimal |
| Приоритет операций | *, /, ^ перед +, -, скобки, ассоциативность |
| Унарный минус | В начале, после оператора, в скобках |
| Степень | 5^0, 5^1, 2^(-2), дробный показатель |
| Негативные кейсы | Деление на ноль, переполнение, ошибки формата |
| История | Добавление, cap на 200 записей, персистентность |
| Настройки темы | Получение, обновление, персистентность, null |

### 7.2 Изоляция тестов

Каждый тест создаёт изолированный экземпляр CalculatorEngine с временными файлами в уникальной папке в %TEMP%:

```
private static CalculatorEngine CreateEngine()
{
    var root = Path.Combine(Path.GetTempPath(),
        "VakhrushevCalculatorTests", Guid.NewGuid().ToString("N"));
    Directory.CreateDirectory(root);
    return new CalculatorEngine(
        historyFilePath:  Path.Combine(root, "history.json"),
        settingsFilePath: Path.Combine(root, "settings.json"));
}
```

Такой подход гарантирует полную изоляцию между тестами: каждый тест работает с собственным набором файлов и не зависит от состояния других тестов или файловой системы.

### 7.3 Результаты тестирования

Все 42 теста успешно проходят. Запуск осуществляется через Test Explorer в Visual Studio 2022 или командой:

```
dotnet test VakhrushevCalculator.sln -c Release
```

Итого: 42 тест-кейса в 19 тестовых методах, 0 ошибок, 0 пропущенных.

## 8. Публикация приложения

Приложение опубликовано как self-contained single-file исполняемый файл для платформы Windows x64. Это означает, что для запуска не требуется установка .NET Runtime на целевом компьютере.

Параметры публикации:

- Configuration: Release
- Target framework: net8.0-windows
- Deployment mode: Self-contained
- Target runtime: win-x64
- Produce single file: включено

Публикация выполняется через меню Build → Publish в Visual Studio 2022 или командой:

```
dotnet publish VakhrushevCalculatorApp\VakhrushevCalculatorApp.csproj \
    -c Release -r win-x64 --self-contained true \
    -p:PublishSingleFile=true
```

## 9. Размещение на GitHub

Исходный код проекта размещён в публичном репозитории на GitHub.

Репозиторий содержит:

- Исходный код всех трёх проектов
- README.md с описанием проекта и инструкцией по запуску
- LICENSE — лицензия MIT
- Release v1.0.0 с готовым исполняемым файлом для Windows x64

Для инициализации репозитория и публикации первой версии использовались команды:

```
git init
git add .
git commit -m "first version"
git branch -M main
git remote add origin https://github.com/<login>/VakhrushevCalculator.git
git push -u origin main
```

## 10. Заключение

В ходе практической работы было разработано настольное приложение «PocketCalc» на платформе .NET 8 с использованием технологии WPF. Все требования задания выполнены:

- Реализованы арифметические операторы +, -, *, /, ^ и скобки
- Логика вынесена в отдельную библиотеку классов VakhrushevCalculatorLib
- Интерфейс реализован в виде WPF-приложения по паттерну MVVM
- Написаны модульные тесты с использованием xUnit (42 теста)
- Проект приложения: VakhrushevCalculatorApp
- Исходный код и сопроводительная записка размещены на GitHub

Дополнительно реализованы: предпросмотр результата в реальном времени, тёмная и светлая тема с сохранением настроек, умная очистка последнего токена (CE), ввод с клавиатуры. Приложение опубликовано как self-contained исполняемый файл для Windows x64.